



Assignment 02: Project AeroAssist

Building Your First AI Support Assistant

Course: Agentic AI: From Concepts to Practice

Instructor: Prof. Karthik Vaidhyanathan

TAs: Aviral Gupta, Aneetta Sara Shany, Ch Pavan Harshit, Shreyash Chandak

Topic: From Chatbots to Local AI

Deadline: 26th July 2026, 11:59 PM

Mission Briefing

TO: AI Systems Engineer (formerly Traffic Analyst)

FROM: Head of Operations, AeroWing Airlines

SUBJECT: Internal Customer Support Initiative

Congratulations on successfully completing the TrafficGuard project. Your solution demonstrated that carefully designed software can solve real operational problems without requiring enormous amounts of data or computing power.

The Operations team has a new challenge.

Every day, our customer support executives answer hundreds of repetitive questions.

- What is the baggage allowance?
- Can I change my booking?
- Does the A320 have Wi-Fi?
- What is the seating capacity of the aircraft?

Some questions are answered correctly.

Some are answered inconsistently.

Some require the agent to search through technical manuals before replying.

Your task is to build an internal AI-powered assistant that helps support executives answer these questions quickly and consistently.

Like every engineering project, this system will evolve over time. You will begin with a cloud-hosted language model, understand how these models behave, experiment with different prompting strategies, and finally deploy a smaller model locally using Ollama.

Learning Objectives

By the end of this assignment you should be able to

- Build a simple chatbot interface using Flask.
- Connect a web application to Google's Gemini API.
- Understand how different language models respond to the same prompt.
- Compare cloud-hosted and locally hosted language models.
- Observe the effect of temperature on generated responses.
- Deploy, switch and interact with a local language model using Ollama.

Assignment Tasks

Part 1: Building the Chatbot

Objective: Create a complete chatbot interface and expose it from Google Colab.

1. Complete the HTML page so it contains a chat history window, a text input, and a send button.
2. Implement the `addMessage` JavaScript function and the `fetch` call that sends the user's message to `/chat` and displays the response.
3. Launch your application using Colab's port forwarding utility and verify that your chatbot can successfully communicate with the backend.

Exploration (Not Graded)

Improve the appearance of your chatbot using HTML and CSS. Consider colours, spacing, fonts and alignment. Your interface does not need to look identical to popular chat applications, but it should be pleasant and easy to use.

Part 2: Connecting Gemini

Objective: Replace the placeholder response with answers generated by Gemini.

1. Stop the running Flask server.
2. Configure the Gemini client using your API key.
3. Update the `/chat` endpoint so that every user message is sent to Gemini.
4. Return the generated response as JSON.
5. Relaunch the application.
6. Test your chatbot using a variety of questions related to airline operations as well as general knowledge.

Critical Incident Report

The prototype is functional and customer feedback has been encouraging. However, several members of the engineering team noticed something interesting. Although the chatbot usually provides useful answers, the responses sometimes vary between runs, occasionally contain unnecessary details, and are not always written in the style expected from an airline support assistant. Before deploying the system, we need to better understand how language models behave and how engineers can influence their responses.

Part 3: Understanding Language Models

Objective: Compare cloud and local language models and investigate how model parameters influence generated responses.

Before attempting to improve the chatbot, it is important to understand the behaviour of the underlying language model. In this section you will compare two different models and observe how changing generation parameters affects their outputs.

1. Comparing Cloud and Local Models

Install Ollama and download one of the language models demonstrated during the tutorials.

Ask both Gemini and your Ollama model the same five customer support questions provided in the notebook.

How did the two models differ in speed and level of detail? Which gave more accurate answers to the airline questions? Which would you rather put in front of a customer, and why?

2. Exploring Temperature

Language models do not always generate the same response to the same prompt. One factor that affects this behaviour is the sampling temperature.

Choose a single prompt and generate responses using (trying absurd prompts might get you interesting results!)

- Temperature = 0
- Temperature = 0.5
- Temperature = 1.0

3. Prompting Strategies

(a) **Zero-shot Prompting** Choose one customer support task. For example,

- classify a customer request,
- summarise a complaint,
- rewrite a response in a professional tone.

Ask the model to perform the task without providing any examples.

(b) **One-shot Prompting** Repeat the same task. This time, provide exactly one worked example before asking your question.

- (c) **Few-shot Prompting** Repeat the experiment once more. Provide at least three examples before asking the model to complete the same task.

4. System Prompts

A system prompt is an instruction given to the model before the conversation starts. It defines the role, tone, and focus of every response that follows. Four role prompts are provided in the notebook. Keep the user question fixed and run it with each role.

5. Providing Additional Context

Language models are trained on general knowledge. They do not know AeroWing's specific aircraft or internal policies.

AeroWing's quick-reference document is provided in the notebook. Ask the same question with and without it included in the prompt.

Critical Incident Report

The Operations team has now tested both Gemini and the local language model. Both systems answer customer questions reasonably well, but the Finance department has raised another concern.

Every customer query is still being answered using the cloud API. Even simple greetings and frequently asked questions require an internet connection and consume paid API requests.

Your manager believes that routine conversations can be handled locally, while the cloud model should remain available whenever higher quality responses are required. Your final task is to integrate both models into a single chatbot that can be deployed across the organisation.

Part 4: Deploying AeroAssist

Objective: Deploy a complete customer support assistant capable of switching between a cloud-hosted and a locally hosted language model.

You have now built every individual component required for deployment.

The final task is to wire them together into a single application that can be used by AeroWing's customer support team.

1. Write a Unified Routing Function

Complete the `ask` function that routes every request to either `askGemini` or `askLocalModel` depending on a configuration variable. Both helper functions are already set up in the notebook.

2. Write a System Prompt

Write a system prompt for AeroAssist. It should tell the model what airline it works for, what kinds of questions it should handle, and how it should behave when it does not know something.

3. Set the Active Model

A configuration variable is provided. Change it to switch between Gemini and Ollama. Nothing else in the application should need to change.

4. Update the Chat Endpoint

Update the `/chat` endpoint to call `ask` with your system prompt. Add a `/model` route so the frontend can display which model is currently active.

Final Report

Answer the following questions in the notebook. Each answer should be three to five sentences.

1. Compare cloud-hosted and locally hosted language models. Discuss one advantage and one limitation of each.
2. Suppose AeroWing plans to deploy specialised assistants for Customer Support, Flight Operations, and Aircraft Maintenance. Would you use the same system prompt for every assistant or design a different one for each role? Explain your reasoning.
3. Which of the experiments in Part 3 had the greatest influence on the quality of the responses? Support your answer with observations from your own experiments.

Submission Guidelines

- Submit a single zip file consisting of
 - A Google Colab notebook (`.ipynb`) containing all code and markdown answers.
 - Screenshots of your chatbot app taken after Part 1 and Part 4. (Titled `partX.jpeg` or `partX.png`)
- Store your Gemini API key securely using Colab Secrets. **DO NOT** leave your key hard-coded in the notebook.
- Submit the file using the filename:

`AeroAssist_YourName.zip`

A language model provides intelligence.

Prompt engineering provides direction.

Thoughtful engineering brings both together into a reliable AI assistant.